

武汉软件工程职业学院

《Web 安全开发》课 程 设 计 报 告

(计算机学院)

题目：轻量级个人博客系统的设计与实现

班级：信安 2401

姓名：陈桂涛

指导教师：陶武婷

2025-2026 学年度第 1 学期

目录

- 1. 需求分析.....3
- 2. 拓扑设计..... 3
 - 2.1 实验目标.....3
 - 2.2 实验功能结构.....4
- 3. 详细实验设计及实验代码..... 4
 - 3.1 首页展示功能..... 4
 - 3.1.1 导航栏设计4
 - 3.1.2 搜索功能实现.....6
 - 3.1.3 文章列表渲染.....9
 - 3.1.4 侧边栏功能..... 12
 - 3.2 文章详情页功能..... 16
 - 3.2.1 Markdown 渲染..... 16
 - 3.2.2 评论系统..... 19
 - 3.3 分类与标签功能..... 24
 - 3.3.1 分类筛选.....24
 - 3.3.2 标签筛选.....26
 - 3.4 后台管理系统..... 28
 - 3.4.1 身份验证.....28
 - 3.4.2 文章管理.....32
 - 3.4.3 分类管理.....38
 - 3.4.4 标签管理.....41
 - 3.4.5 评论管理.....44
 - 3.4.6 系统设置.....46
 - 3.5 数据存储机制..... 52
 - 3.6 样式设计..... 52
- 4. 设计总结..... 55
- 参考文献.....55

轻量级个人博客系统的设计与实现

摘要： 本报告详细阐述了一个轻量级个人博客系统的设计与实现过程。该系统采用纯前端技术栈, 基于 HTML5、CSS3 和 JavaScript 构建, 无需后端服务器支持, 通过浏览器的 localStorage 实现数据持久化存储。系统包含完整的前台展示和后台管理功能, 支持文章发布、Markdown 渲染、分类标签管理、评论互动、全文搜索等核心功能。系统采用模块化设计思想, 前台展示部分包括首页、文章详情页、分类页和标签页, 为访客提供清晰友好的阅读体验; 后台管理系统提供了文章管理、分类管理、标签管理、评论管理和系统设置等功能, 支持博主便捷地管理博客内容。在技术实现上, 系统使用 marked.js 实现 Markdown 解析, highlight.js 提供代码高亮, 采用响应式布局适配不同设备, 通过 CSS3 动画增强用户交互体验。本系统具有轻量级、易部署、零成本的特点, 无需购买服务器和域名, 只需将静态文件部署到 GitHub Pages 等免费托管平台即可使用。整个系统代码结构清晰, 注释完善, 适合作为 Web 前端开发的学习案例。

关键词： 个人博客; 前端开发; localStorage; Markdown; 响应式设计

1. 需求分析

在当今信息化时代, 个人博客作为知识分享和交流的重要平台, 已成为开发者展示技术能力、记录学习心得的首选方式。传统博客系统往往需要配置复杂的服务器环境和数据库系统, 这对初学者和个人用户来说具有较高的技术门槛和维护成本。本项目旨在设计并实现一个轻量级的纯静态个人博客系统, 该系统无需服务器端支持, 仅使用 HTML、CSS 和 JavaScript 前端技术, 通过浏览器的 localStorage 实现数据持久化存储。系统具备完整的内容管理功能, 包括文章发布、分类管理、标签系统、评论互动、搜索功能等, 满足个人博客的基本需求。

2. 拓扑设计

2.1 实验目标

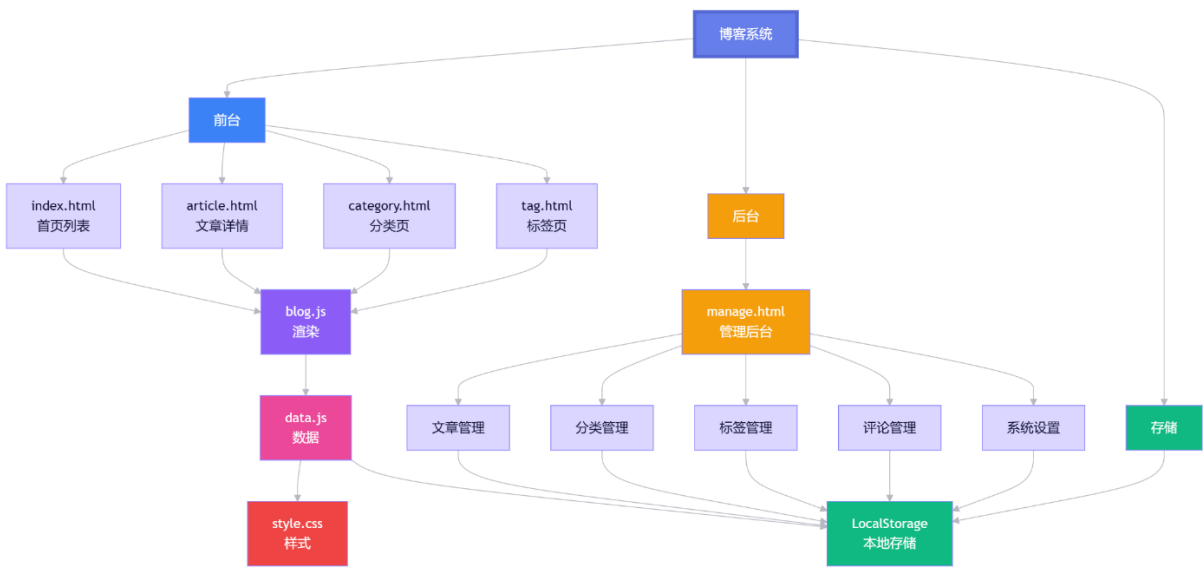
本实验旨在通过纯前端技术实现一个功能完整的个人博客系统, 具体目标包括:

1. 掌握 HTML5 语义化标签的应用
2. 深入理解 CSS3 布局技术
3. 熟练运用 JavaScript 进行 DOM 操作
4. 掌握 localStorage 数据存储机制
5. 实践 Markdown 解析与渲染

6. 构建完整的前端应用架构

2. 2实验功能结构

根据以上需求分析，功能结构如图所示



3. 详细实验设计及实验代码

3. 1 首页展示功能

3. 1. 1 导航栏设计

导航栏是博客系统的核心导航组件,承担着引导用户访问各个功能模块的重要职责。本系统的导航栏设计充分考虑了用户体验和视觉美观性,采用了现代化的设计理念。

思路与实现原理:

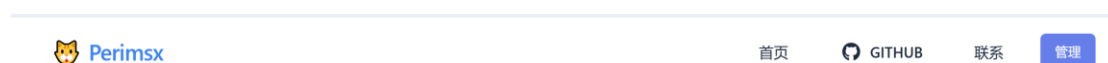
导航栏采用 sticky 固定定位策略,这意味着当用户向下滚动页面时,导航栏会始终保持在视口顶部,确保用户随时可以访问导航功能,无需滚动回页面顶部。这种设计大大提升了用户体验,特别是在浏览长文章时尤为重要。

导航栏的视觉效果采用了半透明背景配合 backdrop-filter 模糊效果,这种设计不仅美观现代,还能让用户在滚动页面时看到背景内容的虚化效果,增强了页面的层次感和动态性。导航栏底部添加了细微的阴影效果,使其与页面主体内容产生视觉分离,更加突出导航区域。

网站 Logo 部分支持多种显示方式,包括 Emoji 表情、自定义图片、Font Awesome 图标和纯文字。这种灵活的设计允许用户根据自己的喜好自定义博客的视觉标识,体现了系统的高度可定制性。Logo 的配置信息存储在 localStorage 中,通过 JavaScript 动态加载,实现了即时生效的效果。

导航链接部分包含了首页、GitHub 链接、联系方式和后台管理四个主要入口。每个链接都添加了悬停效果,当鼠标移入时会显示背景色变化和颜色高亮,为用户提供清晰的视觉反馈。特别值得一提的是管理按钮,采用了渐变色背景和阴影效果,使其在导航栏中更加醒目,方便博主快速进入后台管理界面。

效果:



HTML 结构:

```
<!-- 网站头部导航栏，固定在页面顶部 -->
<header class="nav-fixed">
  <nav>
    <!-- 网站 logo 区域 -->
    <div class="logo" id="siteLogo">Perimsx</div>
    <!-- 导航菜单列表 -->
    <ul>
      <!-- 首页链接 -->
      <li><a href="index.html">首页</a></li>
      <!-- GitHub 链接，使用 Font Awesome 图标 -->
      <li><a href="https://github.com/Perimsx"
class="github-link">
        <i class="fa-brands fa-
github"></i>GITHUB</a></li>
      <!-- 联系我们链接，点击触发 showContact 函数 -->
      <li><a href="#" onclick="showContact()">联系
</a></li>
      <!-- 管理页面链接 -->
      <li><a href="manage.html" class="manage-btn">管理
</a></li>
    </ul>
  </nav>
</header>
```

JavaScript 动态 Logo 加载:

```
function loadSiteIcon() {
  // 从本地存储获取图标值，如果不存在则使用默认值'🐱'
  const icon = localStorage.getItem('site_icon') || '🐱';
```

```

// 从本地存储获取图标类型，如果不存在则使用默认值'emoji'
const iconType = localStorage.getItem('site_icon_type') ||
'emoji';
// 获取网站 logo 元素
const siteLogo = document.getElementById('siteLogo');

// 检查 logo 元素是否存在
if (siteLogo) {
    let iconHtml = '';
    // 如果图标类型是 emoji
    if (iconType === 'emoji') {
        iconHtml = `<span style="margin-
right:8px">${icon}</span>`;
    } else if (iconType === 'image') {
        iconHtml = ``;
    } else if (iconType === 'fontawesome') {
        iconHtml = `<i class="fa ${icon}"></i>`;
    }
    siteLogo.innerHTML = iconHtml + 'Perimsx';
}
}

```

3.1.2 搜索功能实现

搜索功能是博客系统提升用户体验的关键功能之一，它允许访客快速定位感兴趣的内容，而无需逐页浏览。本系统实现了一个功能强大且用户友好的全文搜索系统。

功能设计理念：

搜索框位于导航栏正下方，采用了独立的搜索区域设计，这样的布局使搜索功能更加醒目，用户可以在任何页面上快速找到搜索入口。搜索框采用了圆角设计，配合渐变背景和模糊效果，整体视觉效果现代且优雅。

搜索功能的核心特性是实时搜索，这意味着用户每输入一个字符，系统就会立即进行搜索并更新结果列表。这种即时反馈机制极大地提升了搜索效率，用户无需按回车或点击搜索按钮，就能看到匹配的文章。实时搜索的实现基于 JavaScript 的 input 事件监听，每次输入变化都会触发搜索逻辑。

搜索范围的设计充分考虑了用户的实际需求。系统支持对文章的多个字段进行搜索，包括标题、正文内容、文章简介、分类名称、标签名称和作者名称。这种多字段搜索确保了用户无论从哪个维度查找内容，都能获得准确的结果。例如，用户可以搜索"JavaScript"找到所有与 JavaScript 相关的文章，无论这个关键词出现在标题、内容还是标签中。

搜索算法采用了模糊匹配策略,使用 JavaScript 的 includes() 方法实现部分字符串匹配。为了提供更好的用户体验,搜索时会将所有文本转换为小写进行比较,这样用户无论输入大写还是小写,都能得到一致的搜索结果。这种大小写不敏感的设计符合用户的使用习惯,避免了因大小写不匹配而漏掉结果的情况。

搜索框右侧提供了清除按钮,当用户输入内容后,这个按钮会自动显示。点击清除按钮可以一键清空搜索内容并重置文章列表,这个细节设计让用户操作更加便捷。清除按钮采用了圆形设计,配合旋转动画,增加了交互的趣味性。

视觉交互优化:

搜索框的视觉设计充分考虑了交互状态的反馈。默认状态下,搜索框显示浅色边框和渐变背景;当用户聚焦搜索框时,边框颜色会变为主题蓝色,并显示外发光效果,同时搜索框会有轻微的上移动画,这些细节为用户提供了清晰的操作反馈。搜索图标在聚焦状态下透明度会增加,颜色也会变得更加鲜艳,进一步强化了交互感知。

效果:



HTML 搜索框:

```
<!-- 搜索容器 -->
<div class="search-container">
  <!-- 搜索图标 -->
  <i class="fa fa-search search-icon"></i>
  <!-- 搜索输入框 -->
  <input type="text" id="searchInput"
    placeholder="搜索文章标题、内容、分类、标签...">
  <!-- 清除搜索按钮 -->
  <button class="search-clear" id="searchClear">
    <!-- 关闭图标 -->
```

```
        <i class="fa fa-times"></i>
      </button>
    </div>
```

JavaScript 搜索逻辑:

```
searchInput.addEventListener('input', function() {
  // 获取搜索词并去除首尾空格，转换为小写
  const searchTerm = this.value.trim().toLowerCase();

  // 根据是否有搜索词来显示或隐藏清除按钮
  searchClear.style.display = searchTerm ? 'flex' : 'none';

  // 默认显示所有文章
  let filteredArticles = articles;

  // 如果有搜索词，则进行过滤
  if (searchTerm) {
    // 过滤文章，检查标题、内容、分类和标签是否包含搜索词
    filteredArticles = articles.filter(article => {
      // 检查标题是否匹配
      const titleMatch =
article.title.toLowerCase().includes(searchTerm);
      // 检查内容是否匹配
      const contentMatch =
article.content.toLowerCase().includes(searchTerm);
      // 检查分类是否匹配
      const categoryMatch =
article.category.toLowerCase().includes(searchTerm);
      // 检查标签是否匹配（使用可选链操作符防止标签未定义）
      const tagsMatch = article.tags?.some(tag =>
        tag.toLowerCase().includes(searchTerm));

      // 如果任一条件匹配，则保留该文章
      return titleMatch || contentMatch || categoryMatch
|| tagsMatch;
    });
  }

  // 渲染过滤后的文章列表
  blogSystem.renderArticleList(filteredArticles,
articleList);
});
```


3.1.3 文章列表渲染

文章列表的渲染采用了动态生成的方式,而非静态 HTML。系统在页面加载时,首先从 localStorage 读取文章数据,如果本地没有数据,则使用 data.js 中定义的默认文章作为初始数据。读取到数据后,系统会对文章进行排序,按照发布日期从新到旧排列,确保最新的文章显示在最前面。

每篇文章都以卡片(card)的形式呈现,卡片设计遵循 Material Design 设计规范,具有清晰的层次结构和优雅的阴影效果。卡片分为三个主要部分:文章头部(包含标题、元信息、标签)、文章简介和文章尾部(包含阅读全文链接)。这种结构化的设计使得信息层次分明,用户可以快速浏览和定位感兴趣的内容。

元信息展示设计:

文章元信息包含了丰富的内容,每个元信息项都配有对应的 Font Awesome 图标,使界面更加直观和美观。具体包括:

1. **发布日期:** 显示文章的创建时间,采用中文格式(如"2025 年 1 月 15 日"),符合中文用户的阅读习惯。日期信息帮助用户判断文章的时效性。
2. **作者信息:** 显示文章作者的姓名,虽然个人博客通常只有一个作者,但这个设计为多作者博客提供了扩展可能。
3. **评论数量:** 实时显示该文章的评论总数,这个数字从 localStorage 动态读取,每次有新评论时都会自动更新。评论数为 0 时显示"0 条评论",有助于激发读者参与讨论的兴趣。
4. **浏览次数:** 记录文章被访问的总次数,每次用户打开文章详情页,浏览量自动加 1。这个指标可以帮助用户了解文章的受欢迎程度。
5. **文章字数:** 系统会智能统计文章的字数,这个功能考虑了 Markdown 格式的特殊性,会先移除所有 Markdown 标记(如标题符号、粗体标记、代码块等),然后统计纯文本的字数。对于中文字符,每个字算 1 个字;对于英文单词,每 5 个字母算 1 个字。这种统计方式更符合实际阅读感受。

标签系统设计:

每篇文章可以拥有多个标签,标签以彩色胶囊形式显示在文章标题下方。标签的背景色采用浅蓝色,边框为深蓝色,鼠标悬停时会有轻微的上浮动画效果。点击标签可以快速筛选出所有包含该标签的文章,实现了标签导航功能。标签数据在渲染时会进行安全处理,确保即使数据格式不正确也不会导致页面崩溃。

文章简介功能:

文章简介区域显示文章的 description 字段内容,这是一段精心编写的文章摘要,通常控制在 100-200 字之间,帮助用户快速了解文章主题。如果文章没有设置简介,系统会显示"暂无描述"的提示文字。简介文字采用适中的行高和字号,确保良好的可读性。

阅读全文按钮:

每个文章卡片底部都有一个"阅读全文"按钮,这是用户进入文章详情页的主要入口。按钮采用了现代化的设计,默认状态下显示为透明背景,鼠标悬停时会显示背景色和文字颜色变化,提供清晰的交互反馈。点击按钮会跳转到 `article.html` 页面,并通过 URL 参数传递文章 ID。

性能优化考虑:

为了提升页面加载性能,文章列表采用了一次性渲染的方式,使用数组的 `map` 方法将所有文章转换为 HTML 字符串,最后一次性插入到 DOM 中。这种方式避免了多次 DOM 操作带来的性能开销。同时,文章列表支持延迟加载图片(如果文章包含封面图),使用 `loading="lazy"` 属性实现。

效果:



BlogSystem 类的 `renderArticleList` 方法:

```
renderArticleList(articles, container) {
  // 如果容器不存在, 直接返回
  if (!container) return;

  // 如果文章列表为空, 显示"暂无文章"提示
  if (articles.length === 0) {
    container.innerHTML = '<div class="no-articles"><p>暂无文章</p></div>';
    return;
  }

  // 将文章数组转换为 HTML 字符串并设置到容器中
  container.innerHTML = articles.map(article => {
    // 确保 tags 是数组格式
```

```

        let tags = Array.isArray(article.tags) ? article.tags :
[];
        // 计算文章字数
        const wordCount =
this.calculateWordCount(article.content || '');

        return `
        <article class="blog-post">
            <div class="post-header">
                <h2 class="post-title">
                    <a
href="article.html?id=${article.id}">${article.title}</a>
                </h2>
                <div class="post-meta">
                    <span><i class="fa fa-calendar"></i>
                        ${this.formatDate(article.date)}</span>
                    <span><i class="fa fa-user"></i>
                        ${article.author}</span>
                    <span><i class="fa fa-comments"></i>
                        ${article.comments || 0} 条评论</span>
                    <span><i class="fa fa-eye"></i>
                        ${article.views || 0} 次浏览</span>
                    <span><i class="fa fa-font"></i>
                        ${wordCount} 字</span>
                </div>
                <div class="post-tags">
                    ${tags.map(tag => `<span
class="tag">${tag}</span>`).join('')}
                </div>
            </div>
            <div class="post-content">
                <p>${article.description || '暂无描述'}</p>
            </div>
            <div class="post-footer">
                <a href="article.html?id=${article.id}"
class="read-more">
                    阅读全文
                </a>
            </div>
        </article>`;
    }).join(''); // 将数组中的所有 HTML 字符串连接成一个字符串
}

```

文章字数统计:

效果：

关于我



北京时间
13:49:24

你好，我是Perimsx，一名热爱技术的全栈开发者。在这里分享我的学习心得和开发经验。

文章分类模块：

文章分类模块以列表形式展示所有分类及其文章数量。这个模块的设计充分考虑了用户的浏览习惯，将分类按照文章数量从多到少进行排序，使用户能够优先看到内容最丰富的分类。每个分类项都是可点击的链接，点击后会跳转到分类筛选页面，显示该分类下的所有文章。

分类列表的实现采用了动态统计的方式，系统会遍历所有文章，计算每个分类包含的文章数量，然后将统计结果实时显示在分类名称后面的括号中(如"前端(5)")。这种设计不仅直观地展示了各分类的内容规模，也为用户提供了快速筛选的依据。当后台添加或删除文章时，分类列表会自动更新统计数字，无需手动维护。

分类项的样式设计简洁明了，采用下边框分隔，鼠标悬停时会有左侧缩进和颜色高亮效果，为用户提供清晰的交互反馈。点击分类链接时，URL 会携带分类名称参数，通过 encodeURIComponent 进行编码以支持中文分类名。

效果：

文章分类	
前端	(2)
后端	(2)
常用软件使用	(1)
工具推荐	(1)

热门标签模块：

热门标签模块以标签云的形式展示所有标签，这种展示方式比列表更加活泼和直观。标签采用胶囊形状设计，带有彩色背景和边框，鼠标悬停时会有上浮动画效

果,增强了视觉吸引力。每个标签都添加了 float 动画,使标签云具有轻微的浮动效果,让整个模块更有生命力。

效果:



标签的排列采用 flexbox 弹性布局,自动换行适应容器宽度,确保在不同屏幕尺寸下都能良好显示。点击标签同样会跳转到标签筛选页面,显示所有包含该标签的文章。标签系统支持一篇文章关联多个标签,这种多对多的关系使得内容组织更加灵活,用户可以从多个角度检索文章。

侧边栏定位策略:

侧边栏采用了 sticky 粘性定位策略,这意味着当用户向下滚动页面时,侧边栏会跟随滚动,但始终保持在视口中的固定位置 (top: 100px),这个位置恰好在导航栏下方。这种设计确保了侧边栏的导航功能始终可用,用户无论浏览到页面的哪个位置,都可以快速访问分类和标签。

在响应式设计方面,当屏幕宽度小于 900px 时,侧边栏会自动切换为移动端布局,从右侧栏变为页面底部的横向滚动区域,确保在手机等小屏设备上也能正常使用

北京时间显示:

```
function updateBeijingTime() {
  // 获取当前本地时间
  const now = new Date();
  // 将本地时间转换为北京时间 (Asia/Shanghai 时区)
  const beijingTime = new Date(now.toLocaleString('en-US',
    {timeZone: 'Asia/Shanghai'}));

  // 获取小时并格式化为两位数, 不足补 0
  const hours = String(beijingTime.getHours()).padStart(2,
    '0');
```

```

    // 获取分钟并格式化为两位数，不足补 0
    const minutes =
String(beijingTime.getMinutes()).padStart(2, '0');
    // 获取秒数并格式化为两位数，不足补 0
    const seconds =
String(beijingTime.getSeconds()).padStart(2, '0');

    // 将格式化后的时间更新到页面元素中
    document.getElementById('beijingTime').textContent =
        `${hours}:${minutes}:${seconds}`;
}

// 每隔 1000 毫秒（1 秒）调用一次 updateBeijingTime 函数，实现实时更新
setInterval(updateBeijingTime, 1000);
// 立即调用一次 updateBeijingTime 函数，避免等待 1 秒的延迟
updateBeijingTime();

```

头像加载:

```

function loadAvatar() {
    // 从 localStorage 中获取存储的头像 URL，如果不存在则使用默认路
    径'img/01.jpg'
    const avatarUrl = localStorage.getItem('blog_avatar') ||
'img/01.jpg';
    // 获取 ID 为'avatarImg'的元素，并将其 src 属性设置为获取到的头像
    URL
    document.getElementById('avatarImg').src = avatarUrl;
}

```

分类列表渲染:

```

renderCategories(container) {
    // 获取所有分类数据
    const categories = this.getCategories();
    // 为每个分类添加文章数量统计，并按文章数量降序排序
    const categoriesWithCount = categories.map(cat => {
        // 计算当前分类下的文章数量
        const count = this.articlesData.articles.filter(article
=>
            article.category === cat.name).length;
        // 返回包含分类名称和文章数量的对象
        return {name: cat.name, count};
    }).sort((a, b) => b.count - a.count); // 按文章数量降序排序
}

```



```
// 将分类列表转换为 HTML 字符串并更新容器内容
container.innerHTML = categoriesWithCount.map(cat => `
    <li>
      <a
href="category.html?name=${encodeURIComponent(cat.name)}">
        ${cat.name} <span>(${cat.count})</span>
      </a>
    </li>
` ).join(''); // 将数组转换为字符串
}
```

3. 2文章详情页功能

3. 2. 1 Markdown 渲染

Markdown 是一种轻量级标记语言,因其简洁易学而广受技术博主喜爱。本系统采用 marked.js 库实现 Markdown 到 HTML 的转换,并集成 highlight.js 实现代码高亮,为技术文章提供了专业的展示效果。

文章内容渲染流程:

文章详情页的初始化过程经过精心设计,确保内容的正确展示。首先,系统通过 URL 参数获取文章 ID,这个参数是用户点击文章列表时传递的。如果缺少 ID 参数或 ID 不存在,系统会显示友好的错误提示并提供返回首页的链接。

找到文章后,系统会设置页面标题为"文章标题 - Perimsx",这不仅提升了 SEO 效果,也方便用户在浏览器标签页中识别当前页面。然后进行一个重要的预处理步骤:移除 Markdown 内容中的第一个标题。这是因为文章标题已经在页面头部以 HTML 形式显示,如果不移除 Markdown 中的标题,会导致标题重复显示。系统通过分割文本为行,筛选掉第一个以#开头的行,然后重新拼接来实现。

最后,使用 marked.parse() 方法将预处理后的 Markdown 文本转换为 HTML。这个过程中,marked.js 会识别各种 Markdown 语法(标题、列表、链接、图片、代码块等),并调用 highlight.js 对代码块进行高亮处理。生成的 HTML 包含了完整的语义结构和样式类名,可以直接插入到页面中显示。

Markdown 样式优化:

为了确保渲染后的内容具有良好的可读性,系统为 Markdown 内容设计了专门的 CSS 样式。所有样式都应用到 .post-content 类下,确保只影响文章内容区域:

1. **标题样式:** h1、h2、h3 等标题设置了适当的上下边距和字号,h1 最大(2rem),h3 最小(1.25rem),形成清晰的层次结构。

2. **代码块样式**: 代码块采用深色背景(#1e293b), 配合圆角边框和内边距, 营造了专业的代码展示效果。行内代码则使用浅色背景(#f1f5f9)和红色文字(#e11d48), 与正文区分开来。
3. **表格样式**: 表格采用全宽显示, 设置了边框和内边距, 表头使用深色背景区分, 鼠标悬停时行背景变色, 提升了表格的可读性。
4. **引用样式**: 引用块左侧有蓝色粗边框, 背景为浅灰色, 内边距较大, 使引用内容突出显示。
5. **图片样式**: 图片设置了最大宽度 100%, 高度自适应, 并添加了圆角边框, 确保图片不会超出容器宽度且保持比例。

代码高亮功能:

代码高亮是技术博客的重要特性。系统集成了 highlight.js 库, 支持 200+ 种编程语言的语法高亮。当用户在 Markdown 中使用 **语法创建代码块并指定语言时** (如 javascript), highlight.js 会自动识别语言并应用相应的高亮规则。即使未指定语言, 库也能通过启发式算法自动检测代码类型。

系统选用了 github-dark 高亮主题, 这是 GitHub 暗色主题的经典配色方案, 对关键字、字符串、注释等不同语法元素使用了区分度高的颜色, 使得代码结构一目了然, 极大地提升了代码的可读性。

效果:

Markdown 配置:

```
marked.setOptions({
  // 启用换行, 使得 Markdown 中的换行符能够转换为 HTML 中的<br>标签
  breaks: true,
  // 启用 GitHub 风格 Markdown(GFM), 支持 GitHub 特有的 Markdown 语法
  gfm: true,
  // 启用表格支持, 允许在 Markdown 中使用表格语法
  tables: true,
  // 设置代码高亮函数, 用于高亮显示代码块
  highlight: function(code, lang) {
    // 如果指定了语言且 hljs 支持该语言
    if (lang && hljs.getLanguage(lang)) {
      // 使用指定的语言进行代码高亮
      return hljs.highlight(code, {language: lang}).value;
    }
    // 否则自动检测语言并进行高亮
    return hljs.highlightAuto(code).value;
  }
});
```

文章内容渲染:

```
async function initArticlePage() {
  // 初始化博客系统
  blogSystem.init();
  // 从 URL 参数中获取文章 ID
  const articleId = blogSystem.getUrlParam('id');
  // 根据文章 ID 获取文章内容
  const article = blogSystem.getArticleById(articleId);

  // 移除第一个标题避免重复
  let contentToRender = article.content;
  const lines = contentToRender.split('\n');
  const filteredLines = lines.filter((line, index) => {
    // 如果是第一行且以#开头，则过滤掉该行
    if (index === 0 && line.trim().startsWith('#')) return
false;
    return true;
  });
  contentToRender = filteredLines.join('\n');

  // 将 Markdown 内容转换为 HTML
  const htmlContent = marked.parse(contentToRender);

  // 将文章内容渲染到页面
  document.getElementById('article-detail').innerHTML = `
    <div class="post-header">
      <h1 class="post-title">${article.title}</h1>
      <div class="post-meta">
        <span><i class="fa fa-calendar"></i>
          ${blogSystem.formatDate(article.date)}</span>
        <span><i class="fa fa-user"></i>
          ${article.author}</span>
        <span><i class="fa fa-eye"></i> ${article.views}
浏览</span>
      </div>
    </div>
    <div class="article-content post-
content">${htmlContent}</div>
  `;
}
```

3.2.2 评论系统

评论系统是博客互动的核心功能,它为读者和博主之间搭建了交流的桥梁。本系统实现了一个功能完善且用户友好的评论系统,支持匿名评论、昵称记忆、邮箱验证等特性。

评论表单设计:

评论表单采用了简洁明了的设计,分为三个主要输入区域:昵称、邮箱和评论内容。每个输入框都配有清晰的标签和 Font Awesome 图标,帮助用户快速理解各个字段的作用。

1. **昵称字段(必填):**这是评论者的身份标识,设置了长度限制(2-50 个字符),避免过短或过长的昵称。系统会自动保存用户输入的昵称到 localStorage,下次访问时自动填充,这个贴心的设计避免了用户重复输入的麻烦。
2. **邮箱字段(选填):**邮箱是可选项,但填写后会在评论显示时带上"已验证"徽章,增加评论的可信度。邮箱同样会被保存到 localStorage,实现记忆功能。系统对邮箱格式进行了验证,确保输入的是有效的邮箱地址。
3. **评论内容(必填):**评论文本域支持多行输入,设置了内容长度限制(2-500 个字符),既避免了空评论,也防止了过长内容对页面布局的影响。文本域下方实时显示字符计数("0/500"),这个视觉反馈帮助用户掌握输入长度,避免超出限制。

评论提交逻辑:

评论提交的处理流程经过了精心设计,确保数据的安全性和完整性。当用户点击"发表评论"按钮时,系统会进行以下步骤:

1. **表单验证:**首先检查昵称和评论内容是否为空,然后验证字符长度是否符合要求。如果验证失败,会通过 showAlert() 函数显示友好的错误提示,告诉用户具体的错误原因。
2. **数据保存:**验证通过后,系统会将昵称和邮箱保存到 localStorage,使用 'comment_author' 和 'comment_email' 为 key。这样下次用户访问时,这些信息会自动填充到表单中。
3. **创建评论对象:**系统创建一个包含所有评论信息的对象,包括昵称、邮箱、内容、创建时间、唯一 ID 等。评论 ID 使用 Date.now() 生成,确保每条评论都有唯一标识。创建时间使用 ISO 格式存储,方便后续的日期处理。
4. **存储到 localStorage:**评论数据以文章 ID 为 key 存储在 localStorage 中,格式为 `comments_${articleId}`。这种设计使得每篇文章的评论独立存储,互不干扰。
5. **界面更新:**提交成功后,系统显示成功提示,清空评论内容输入框(但保留昵称和邮箱),并重新加载评论列表,让新评论立即显示出来。

评论列表渲染:

评论列表的展示采用了精美的卡片式设计,每条评论都是一个独立的卡片。评论按照发表时间降序排列,最新的评论显示在最前面。每个评论卡片包含:

1. **头像区域:**采用圆形头像设计,但不是真实图片,而是显示昵称的首字母大写形式。头像的背景颜色根据昵称生成,使用哈希算法确保同一昵称始终使用同一颜色。系统预定义了 12 种渐变色作为头像背景,通过昵称的哈希值对颜色数量取模,实现颜色的分配。
2. **评论头部:**显示评论者昵称、邮箱验证徽章和评论时间。如果评论者填写了邮箱,会显示一个绿色的"已验证"徽章,这个细节设计增加了评论的可信度。
3. **评论内容:**显示评论的正文内容,系统会对内容进行 HTML 转义,防止 XSS 攻击。同时将换行符(\n)转换为
标签,保留用户输入的段落格式。

评论时间智能显示:

评论时间的显示采用了智能化处理,根据时间间隔显示不同的格式:

- 刚发表(60 秒内):显示"刚刚"
- 1 小时内:显示"X 分钟前"
- 24 小时内:显示"X 小时前"
- 7 天内:显示"X 天前"
- 7 天以上:显示具体日期(如"2025 年 1 月 15 日")

这种设计使得近期评论的时间更加直观,而较旧的评论则显示具体日期。系统通过计算当前时间与评论时间的差值,将差值转换为秒、分、时、天等单位,然后根据不同范围返回相应的显示文字。

头像颜色生成算法:

为了让每个评论者都有独特的视觉标识,系统实现了一个巧妙的颜色生成算法。该算法首先定义了 12 种渐变色,然后对昵称字符串进行哈希计算。哈希计算遍历昵称的每个字符,将字符的 Unicode 值累加到哈希值中,并使用位移算法增加散列效果。最后,将哈希值对颜色数量取模,得到颜色索引。这种方法确保同一昵称总是得到相同的颜色,不同昵称则尽可能分散到不同的颜色。

安全性考虑:

评论系统在安全性方面做了充分的防护:

1. **HTML 转义:**所有用户输入的内容在显示前都会通过 `escapeHtml()` 函数进行转义,将<>等特殊字符转换为<>等 HTML 实体,防止 XSS(跨站脚本)攻击。

2. **长度限制**: 对昵称和评论内容设置了长度限制, 防止恶意用户提交过长的内容占用存储空间或破坏页面布局。
3. **输入验证**: 对邮箱格式进行验证, 确保输入的是有效的邮箱地址, 防止无效数据存储。

效果:

昵称 *

12

邮箱 (可选) 邮箱地址

评论内容 *

分享你的想法...支持换行

0/500

发表评论

评论发表成功!

12 刚刚

111

评论表单 HTML:

```
<form id="comment-form">
  <!-- 昵称输入框, 必填项 -->
  <input type="text" id="comment-author" placeholder="昵称"
required>
  <!-- 邮箱输入框, 可选项 -->
  <input type="email" id="comment-email" placeholder="邮箱(可
选)">
  <!-- 评论内容输入框, 必填项 -->
  <textarea id="comment-content" placeholder="评论内容"
required></textarea>
  <!-- 提交按钮 -->
  <button type="submit">发表评论</button>
</form>
```

评论表单 HTML:

```
<!-- 评论表单 -->
```

```

<form id="comment-form">
  <!-- 输入框：用于输入用户昵称 -->
  <!-- type="text" 表示文本输入框 -->
  <!-- id="comment-author" 为该输入框指定唯一标识符 -->
  <!-- placeholder="昵称" 设置输入框提示文本 -->
  <!-- required 表示该字段为必填项 -->
  <input type="text" id="comment-author" placeholder="昵称"
required>
  <!-- 输入框：用于输入用户邮箱 -->
  <!-- type="email" 表示邮箱格式的输入框，会验证输入内容是否符合
邮箱格式 -->
  <!-- id="comment-email" 为该输入框指定唯一标识符 -->
  <!-- placeholder="邮箱(可选)" 设置输入框提示文本 -->
  <!-- 没有 required 属性，表示该字段为可选项 -->
  <input type="email" id="comment-email" placeholder="邮箱(可
选)">
  <!-- 文本域：用于输入评论内容 -->
  <!-- id="comment-content" 为该文本域指定唯一标识符 -->
  <!-- placeholder="评论内容" 设置文本域提示文本 -->
  <!-- required 表示该字段为必填项 -->
  <textarea id="comment-content" placeholder="评论内容"
required></textarea>
  <!-- 提交按钮：用于提交评论表单 -->
  <!-- type="submit" 表示这是一个提交按钮 -->
  <button type="submit">发表评论</button>
</form>

```

提交评论:

```

function submitComment(articleId) {
  // 获取并去除用户输入的昵称、邮箱和评论内容的前后空格
  const author = document.getElementById('comment-
author').value.trim();
  const email = document.getElementById('comment-
email').value.trim();
  const content = document.getElementById('comment-
content').value.trim();

  // 检查昵称和评论内容是否为空，如果为空则显示警告提示并返回
  if (!author || !content) {
    showAlert('请填写昵称和评论内容', 'warning');
    return;
  }

  // 将用户昵称和邮箱保存到本地存储中
  localStorage.setItem('comment_author', author);
}

```

```

    if (email) localStorage.setItem('comment_email', email);

    // 创建评论对象，包含作者、邮箱和内容
    const comment = {author, email, content};
    // 调用保存评论的函数，传入文章 ID 和评论对象
    saveComment(articleId, comment);

    // 显示评论发表成功的提示
    showAlert('评论发表成功', 'success');
    // 清空评论内容输入框
    document.getElementById('comment-content').value = '';
    // 重新加载并显示该文章的所有评论
    loadComments(articleId);
}

```

评论列表渲染:

```

function loadComments(articleId) {
    // 获取指定文章的评论数据
    const comments = getComments(articleId);
    // 对评论进行排序，按创建时间降序排列（最新的在前）
    const sortedComments = [...comments].sort((a, b) =>
        new Date(b.created_at) - new Date(a.created_at));

    // 将排序后的评论列表渲染到页面的评论区域
    document.getElementById('comments-list').innerHTML =
        sortedComments.map(comment => `
        <div class="comment-item">
            <!-- 用户头像区域 -->
            <div class="avatar-circle"
                style="background:${getAvatarColor(comment.auth
or)}}">

                <!-- 显示用户名首字母作为头像 -->
                ${comment.author.charAt(0).toUpperCase()}
            </div>
            <!-- 评论内容区域 -->
            <div class="comment-body">
                <!-- 用户名 -->
                <strong>${escapeHtml(comment.author)}</strong>
                <!-- 如果用户已验证邮箱，显示验证徽章 -->
                ${comment.email ? '<span class="badge">已验证
</span>' : ''}

                <!-- 评论发布时间 -->
                <span
class="time">${formatCommentDate(comment.created_at)}</span>
                <!-- 评论正文，将换行符转换为 HTML 的<br>标签 -->

```



```

        <p>${escapeHtml(comment.content).replace(/\n/g,
'<br>')}</p>
        </div>
    </div>
    `).join(''); // 将生成的 HTML 片段连接成一个完整的字符串
}

```

3. 3分类与标签功能

3. 3. 1 分类筛选

分类筛选功能允许用户快速查看某一分类下的所有文章。这个功能在 category.html 页面实现,具有简洁清晰的交互逻辑和高效的数据查询性能。

功能实现原理:

分类页面的工作原理非常直接。当用户点击某个分类链接时(比如点击侧边栏中的"前端"分类),浏览器会跳转到 category.html 页面,并在 URL 中携带分类名称参数(如 category.html?name=%E5%89%8D%E7%AB%AF)。页面加载后,JavaScript 会解析 URL 参数获取分类名,然后从文章数据集中筛选出属于该分类的所有文章。

URL 参数处理:

系统使用 getUrlParam() 方法获取 URL 中的参数值。这个方法通过 URLSearchParams API 解析查询字符串,支持中文分类名的自动解码。比如 URL 中的%E5%89%8D%E7%AB%AF 会被自动解码为"前端"。如果缺少 name 参数,页面会显示所有文章,避免页面空白。

文章筛选逻辑:

getArticlesByCategory() 方法负责执行实际的筛选操作。该方法使用 JavaScript 的 filter() 函数遍历所有文章,保留 category 属性与目标分类名完全相等的文章。这里使用的是严格相等(===)比较,确保匹配的准确性。筛选后的文章会按照发布日期降序排列,使用 sort() 方法比较文章的 date 属性,让最新文章显示在前面。

页面标题动态设置:

为了提升用户体验和 SEO 效果,系统会根据当前查看的分类动态设置页面标题。比如查看"前端"分类时,浏览器标签页会显示"前端 - Perimsx",这样用户在多个标签页之间切换时容易区分。标题设置使用 document.title 属性,这个值也会被搜索引擎索引,有利于 SEO 优化。

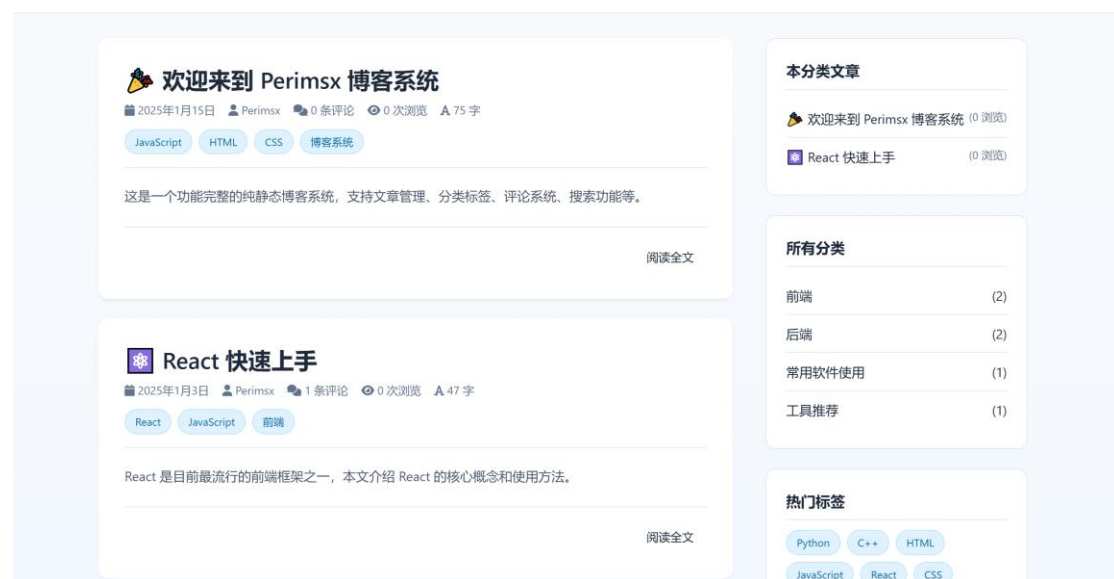
列表渲染复用:

分类页面的文章列表渲染复用了首页的 `renderArticleList()` 方法。这种代码复用不仅减少了代码重复,也确保了界面的一致性。无论在哪个页面,文章卡片的样式、布局和交互都完全相同,给用户提供了统一的使用体验。

空状态处理:

如果某个分类下没有任何文章,`renderArticleList()` 方法会显示一个友好的"暂无文章"提示,而不是留下空白页面。这个细节处理提升了用户体验,让用户明确知道当前状态而不是误以为系统出错。

效果:



分类页面逻辑:

```
/**
 * 当 DOM 内容加载完成后执行的初始化函数
 * 使用 async/await 异步加载博客系统
 */
document.addEventListener('DOMContentLoaded', async () => {
  // 初始化博客系统
  await blogSystem.init();
  // 从 URL 参数中获取分类名称
  const categoryName = blogSystem.getUrlParam('name');

  // 如果存在分类名称,则执行以下操作
  if (categoryName) {
    // 设置页面标题为"分类名称 - Perimssx"
    document.title = `${categoryName} - Perimssx`;
    // 根据分类名称获取相关文章
    const articles =
blogSystem.getArticlesByCategory(categoryName);
```

```

        // 渲染文章列表到指定容器
        blogSystem.renderArticleList(articles, articleList);
    }
});

/**
 * 根据分类名称获取文章列表
 * @param {string} category - 文章分类名称
 * @returns {Array} 返回按日期降序排列的文章数组
 */
getArticlesByCategory(category) {
    // 使用 filter 方法筛选出指定分类的文章
    // 然后使用 sort 方法按日期降序排列
    return this.articlesData.articles.filter(article =>
        article.category === category
    ).sort((a, b) => new Date(b.date) - new Date(a.date));
}

```

3.3.2 标签筛选

标签筛选功能与分类筛选类似,但在实现逻辑上有所不同。由于一篇文章可以拥有多个标签,标签筛选需要检查文章的 tags 数组是否包含目标标签,而不是简单的相等匹配。

功能实现原理:

标签页面(tag.html)的工作流程与分类页面很相似:通过 URL 参数获取标签名,然后筛选包含该标签的所有文章。不同之处在于筛选条件的实现:getArticlesByTag() 方法使用数组的 includes() 方法检查文章的 tags 数组是否包含目标标签。

数组包含判断:

每篇文章的 tags 属性是一个字符串数组,比如['JavaScript', 'React', '前端']。当用户点击'React'标签时,系统需要找出所有 tags 数组中包含'React'字符串的文章。includes() 方法完美地实现了这个需求,它会判断数组中是否存在指定值,返回 true 或 false。

多标签支持:

标签系统的灵活性体现在其多对多的关联方式。同一篇文章可以同时拥有'JavaScript'、'React'和'前端'三个标签,这意味着这篇文章会同时出现在这三个标签的筛选结果中。这种设计使得内容组织更加灵活,用户可以从不同的角度检索同一篇文章。

排序与展示:

与分类筛选一样,标签筛选结果也按照发布日期降序排列,确保最新内容优先展示。页面标题同样会动态设置为当前标签名,比如"React - Perimsx"。文章列表的渲染也复用了 `renderArticleList()` 方法,保持界面一致性。

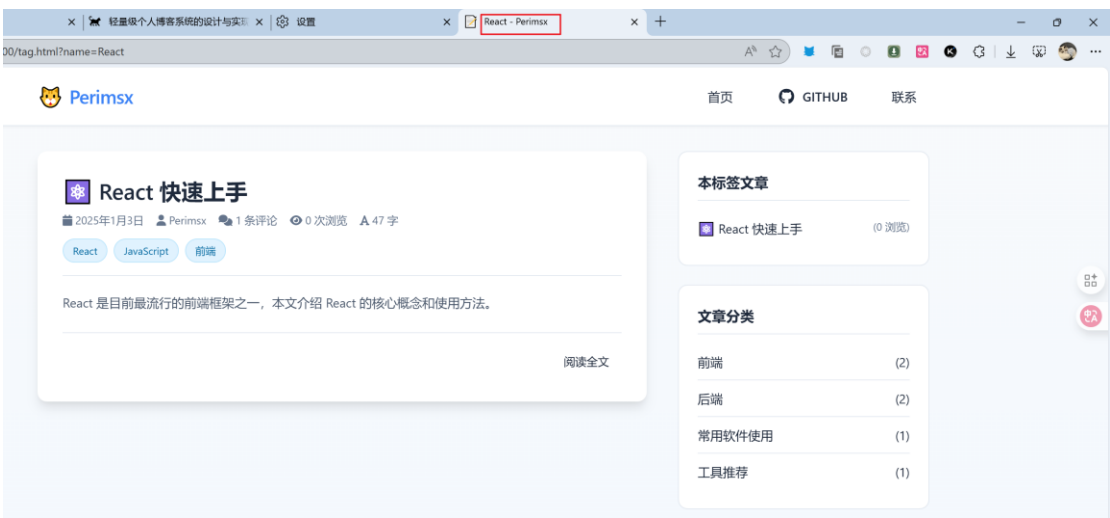
标签云交互:

首页侧边栏的热门标签区域以标签云形式展示所有标签。每个标签都是可点击的链接,点击后跳转到标签页面。标签的视觉设计非常活泼,采用彩色背景和边框,鼠标悬停时有上浮动画,增强了点击的诱导力。

标签统计:

系统会自动统计每个标签被使用的次数。这个功能通过遍历所有文章,收集所有 `tags` 数组中的标签,并计算每个标签出现的次数实现。虽然当前在侧边栏没有显示标签数量,但这个统计数据可以用于未来的功能扩展,比如"热门标签"排序。

效果:



标签页面逻辑:

```
/**
 * DOM 内容加载完成后执行的初始化函数
 * 该函数会初始化博客系统,并根据 URL 参数获取并渲染对应标签的文章列表
 */
document.addEventListener('DOMContentLoaded', async () => {
  // 初始化博客系统
  await blogSystem.init();
});
```

```

// 从 URL 参数中获取标签名称
const tagName = blogSystem.getUrlParam('name');

// 如果存在标签名称参数
if (tagName) {
  // 设置页面标题为"标签名 - Perimsx"
  document.title = `${tagName} - Perimsx`;
  // 获取该标签下的所有文章
  const articles = blogSystem.getArticlesByTag(tagName);
  // 渲染文章列表到指定容器
  blogSystem.renderArticleList(articles, articleList);
}
});

/**
 * 根据标签名称获取文章列表的方法
 * @param {string} tag - 要筛选的标签名称
 * @return {Array} 返回按日期降序排列的文章数组
 */
getArticlesByTag(tag) {
  // 使用 filter 方法筛选包含指定标签的文章
  // 然后按发布日期降序排序
  return this.articlesData.articles.filter(article =>
    article.tags.includes(tag)
  ).sort((a, b) => new Date(b.date) - new Date(a.date));
}

```

3. 4后台管理系统

3. 4. 1 身份验证

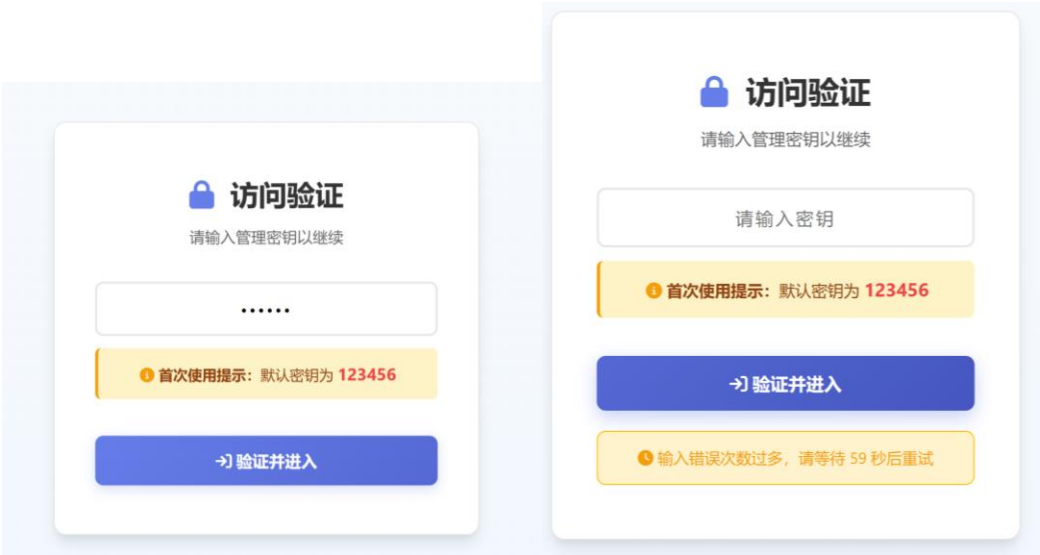
身份验证是后台管理系统的第一道安全屏障,防止未授权用户访问管理功能。系统采用了密钥验证机制,结合失败次数限制和临时锁定,实现了简单而有效的访问控制。

验证界面设计:

当用户访问后台管理页面时,首先会看到一个全屏的验证遮罩层。这个遮罩层采用半透明黑色背景,中间是一个白色的验证卡片。卡片包含锁图标、标题、提示文字、密码输入框和验证按钮。输入框采用 password 类型,输入的密钥会以星

号显示,保护隐私。卡片设计简洁大方,配色以蓝色为主色调,符合现代化的审美。

效果:



密钥验证逻辑:

系统验证逻辑的核心是 `checkAuth()` 函数。这个函数首先检查当前是否处于锁定状态。如果用户之前输入错误过多导致被锁定,且锁定时间还未到期,系统会显示剩余锁定时间,并拒绝验证请求。

如果未被锁定,系统会获取用户输入的密钥,并与 `localStorage` 中存储的正确密钥进行比较。默认密钥为 '123456',用户可以在系统设置中修改。如果密钥匹配,验证成功,系统会清零失败计数器,隐藏遮罩层,用户进入管理后台。

错误处理与锁定机制:

如果密钥不匹配,系统会增加失败计数器(`failedAttempts`),并将计数值保存到 `localStorage`。失败计数器使用 '`auth_failed`' key 存储,确保刷新页面后计数依然有效。

系统允许用户最多错误 5 次。当错误次数小于 5 时,系统会显示错误提示和剩余机会次数,比如"密钥错误,还有 3 次机会"。当错误次数达到 5 次时,系统会触发锁定机制,计算锁定截止时间(当前时间+60 秒),并将其存储到 `localStorage` 的 '`auth_lock_until`' key 中。

锁定期间,系统会显示锁定提示和倒计时。倒计时通过 `setInterval` 实现,每秒更新一次剩余时间,直到锁定时间到期后自动解锁。这种设计有效防止了暴力破解攻击,同时也给予了用户明确的反馈。

持久化存储:

验证状态的持久化存储是安全机制的重要组成部分。系统将失败计数和锁定时间存储在 localStorage 中,这意味着即使用户关闭浏览器或刷新页面,这些状态仍然会保持。攻击者无法通过刷新页面来重置失败计数。只有验证成功后,失败计数才会被清零。

安全性考虑:

虽然这是一个纯前端的验证机制,安全性有限,但对于个人博客来说已经足够。密钥存储在 localStorage 中,虽然可以通过浏览器开发者工具查看,但已经阻挡了大部分普通用户的非法访问。失败次数限制和锁定机制进一步提升了安全性,防止了自动化的暴力破解尝试。如果需要更高的安全等级,建议在后端部署真正的身份验证服务。

密钥验证 HTML:

```
<div id="authOverlay" class="auth-overlay">
  <!-- 认证框 -->
  <div class="auth-box">
    <!-- 标题, 包含锁图标和"访问验证"文字 -->
    <h2><i class="fa fa-lock"></i> 访问验证</h2>
    <!-- 提示信息 -->
    <p>请输入管理密钥以继续</p>
    <!-- 密码输入框 -->
    <input type="password" id="authInput" placeholder="请输入密钥">
    <!-- 验证按钮 -->
    <button onclick="checkAuth()">验证并进入</button>
    <!-- 错误提示信息 -->
    <div id="authError" class="auth-error">密钥错误</div>
    <!-- 锁定提示信息, 包含倒计时 -->
    <div id="authLocked" class="auth-locked">
      输入错误次数过多, 请等待 <span id="lockTime">60</span>
      秒
    </div>
  </div>
</div>
```

验证逻辑:

```
let failedAttempts =
parseInt(localStorage.getItem('auth_failed') || '0');
let lockUntil =
parseInt(localStorage.getItem('auth_lock_until') || '0');
```



```
function checkAuth() {
    const now = Date.now();

    // 检查是否被锁定
    if (lockUntil > now) {
        const remainingSeconds = Math.ceil((lockUntil - now) /
1000);
        showLockMessage(remainingSeconds);
        return;
    }

    const input = document.getElementById('authInput').value;
    const correctKey = localStorage.getItem('admin_key') ||
'123456';

    if (input === correctKey) {
        // 验证成功
        failedAttempts = 0;
        localStorage.setItem('auth_failed', '0');
        document.getElementById('authOverlay').classList.add('h
idden');
    } else {
        // 验证失败
        failedAttempts++;
        localStorage.setItem('auth_failed',
failedAttempts.toString());

        if (failedAttempts >= 5) {
            // 锁定 1 分钟
            lockUntil = now + 60000;
            localStorage.setItem('auth_lock_until',
lockUntil.toString());
            showLockMessage(60);
        } else {
            showError(`密钥错误,还有${5-failedAttempts}次机会`);
        }
    }
}
```

3.4.2 文章管理

文章管理是后台管理系统的核心功能,提供了文章的增、删、改、查全部功能。

系统采用表单式界面,支持 Markdown 格式的文章创作,并提供了完善的文章列表管理功能。

新建文章表单设计:

新建文章表单包含了文章的所有基本信息字段。首先是文章 ID 字段,这是文章的唯一标识符,用于 URL 路由和数据查询。ID 应该使用英文或数字,避免使用中文或特殊字符。

标题字段是必填项,输入框设置了 `required` 属性,确保用户必须填写标题才能提交。作者字段默认填充为 "Perimsx",用户可以修改。发布日期字段采用 HTML5 的 `date` 类型输入框,会弹出日期选择器,方便用户选择日期。

分类字段采用下拉选择框(`select`),选项会动态加载已有的所有分类,确保文章分类的一致性。标签字段采用文本输入框,多个标签用英文逗号分隔,比如 "JavaScript,React,前端"。系统会在提交时自动切分并去除空格。

文章简介字段是可选项,用于在文章列表中显示摘要。最重要的是文章内容字段,这是一个大型文本域(`textarea`),支持多行输入和 Markdown 语法。用户可以在这里使用 Markdown 格式编写文章,包括标题、列表、链接、图片、代码块等。

文章添加逻辑:

`addArticle()` 函数负责处理文章的添加。首先,系统从表单各个字段收集数据,使用 `trim()` 方法去除首尾空格,确保数据的整洁性。特别是标签字段,需要将逗号分隔的字符串转换为数组,使用 `split(',').map(t => t.trim()).filter(t => t)` 实现切分、去空格和过滤空值。

然后,系统构造一个文章对象,包含 `id`、`title`、`author` 等所有属性。`views` 和 `comments` 属性初始化为 0,表示新文章的浏览量和评论数为零。

接着,从 `localStorage` 读取现有的文章数组。如果是第一次使用,`localStorage` 中不存在 `blog_articles` 这个 key,则初始化为空数组。将新文章添加到数组末尾,然后将整个数组序列化为 JSON 字符串并保存回 `localStorage`。

最后,显示成功提示,清空表单,并刷新文章列表,让新添加的文章立即显示出来。`showToast()` 是一个通用的提示函数,会在页面右上角显示一个消息提示条,几秒后自动消失。

效果：

+ 新建文章

≡ 文章列表

📁 分类管理

🏷️ 标签管理

💬 评论管理

⚙️ 系统设置

作者

Perimxs

发布日期 +

2025/11/23

分类 +

前端

标签

多个标签用逗号分隔，例如: JavaScript, 教程, 前端

文章简介

简要描述文章内容...

正文内容 +

使用 Markdown 格式编写文章正文...

示例：
一级标题
二级标题

这是一段文字。

文章编辑功能：

editArticle() 函数实现文章的编辑功能。当用户点击文章列表中的"编辑"按钮时,系统会根据文章 ID 查找对应的文章数据,然后将数据填充到编辑表单中。编辑表单通常以模态弹窗(modal)的形式显示,覆盖在页面上方,背景半透明。

用户修改完成后点击保存,saveArticleEdit() 函数会被调用。该函数首先根据文章 ID 在数组中找到对应的文章,然后更新其属性,最后保存回 localStorage。编辑功能特别适合修正文章中的错别字或更新过时内容。

文章删除功能：

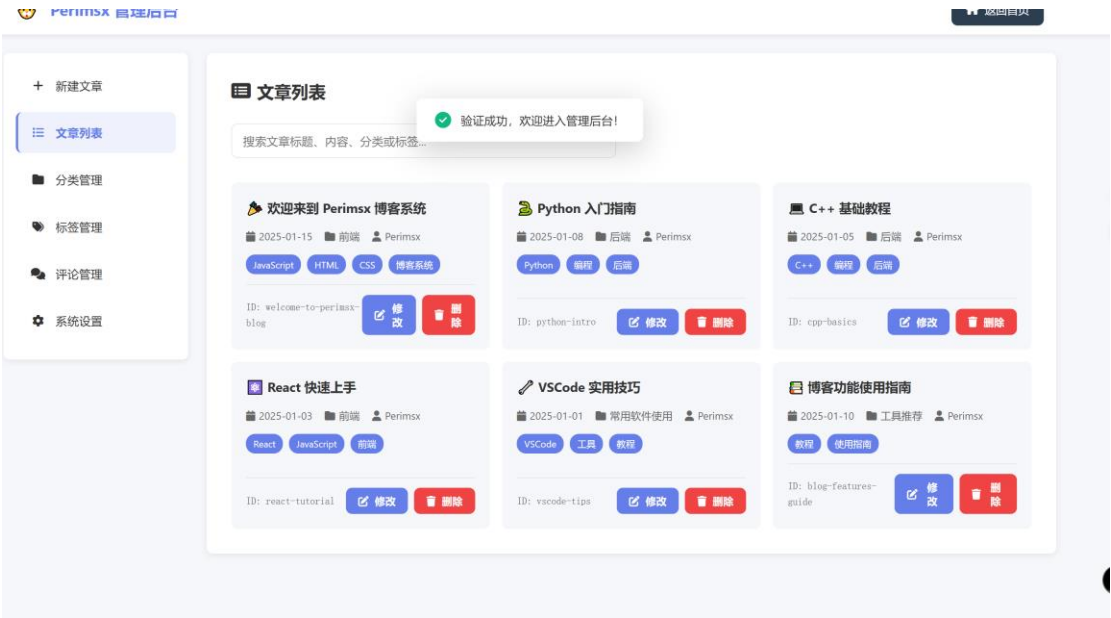
deleteArticle() 函数实现文章的删除。由于删除是不可逆操作,系统会先弹出确认对话框,询问用户是否确定删除。只有用户点击确认后,才会执行实际的删除操作。

删除逻辑使用数组的 filter() 方法,保留所有 ID 不等于目标文章 ID 的文章,相当于从数组中移除目标文章。然后将新数组保存回 localStorage,覆盖原有数据。删除成功后显示提示并刷新文章列表。

文章列表展示：

后台的文章列表以表格形式展示,包含 ID、标题、分类、作者、日期、浏览量、评论数和操作按钮等列。表格采用条纹背景,偶数行和奇数行颜色不同,提升可读性。操作按钮包括"编辑"和"删除",采用不同颜色区分,编辑按钮为蓝色,删除按钮为红色。

效果:



新建文章表单:

```
<!-- 表元素, 用于添加新文章 -->
<form onsubmit="event.preventDefault(); addArticle();">
  <!-- 文章 ID 输入框, 必填项 -->
  <input type="text" id="articleId" placeholder="文章 ID"
required>
  <!-- 文章标题输入框, 必填项 -->
  <input type="text" id="articleTitle" placeholder="标题"
required>
  <!-- 文章作者输入框, 默认值为"Perimx" -->
  <input type="text" id="articleAuthor" value="Perimx">
  <!-- 文章日期选择器, 必填项 -->
  <input type="date" id="articleDate" required>
  <!-- 文章分类下拉选择框, 必填项 -->
  <select id="articleCategory" required></select>
  <!-- 文章标签输入框, 多个标签用逗号分隔 -->
  <input type="text" id="articleTags" placeholder="标签,逗号分
隔">
  <!-- 文章简介文本域 -->
  <textarea id="articleDesc" placeholder="简介"></textarea>
  <!-- 文章内容文本域, 支持 Markdown 格式, 必填项 -->
```

```
    <textarea id="articleContent" placeholder="Markdown 内容"
required></textarea>
    <!-- 提交按钮，用于发布文章 -->
    <button type="submit">发布文章</button>
</form>
```

添加文章逻辑:

```
function addArticle() {
    // 创建文章对象，从表单中获取数据并进行处理
    const article = {
        id: document.getElementById('articleId').value.trim(),
// 文章 ID，去除前后空格
        title:
document.getElementById('articleTitle').value.trim(), // 文章标
题，去除前后空格
        author:
document.getElementById('articleAuthor').value.trim(), // 作者
名，去除前后空格
        date: document.getElementById('articleDate').value, //
发布日期
        category:
document.getElementById('articleCategory').value, // 文章分类
        tags:
document.getElementById('articleTags').value.split(',') // 标签
字符串处理为数组
                .map(t => t.trim()).filter(t => t),
        description:
document.getElementById('articleDesc').value.trim(),
        content:
document.getElementById('articleContent').value.trim(),
        views: 0,
        comments: 0
    };

    // 保存到 localStorage
```

```

    let articles =
JSON.parse(localStorage.getItem('blog_articles') || '[]');
    articles.push(article);
    localStorage.setItem('blog_articles',
JSON.stringify(articles));

    showToast('文章发布成功', 'success');
    document.querySelector('form').reset();
    loadArticleList();
}

```

编辑文章:

```

/**
 * 编辑文章函数
 * @param {number} articleId - 要编辑的文章 ID
 */
function editArticle(articleId) {
    // 从 localStorage 中获取所有文章数据并解析为 JavaScript 对象
    let articles =
JSON.parse(localStorage.getItem('blog_articles'));
    // 根据文章 ID 查找对应的文章对象
    const article = articles.find(a => a.id === articleId);

    // 填充表单
    document.getElementById('editArticleId').value =
article.id;
    document.getElementById('editArticleTitle').value =
article.title;
    document.getElementById('editArticleContent').value =
article.content;

    // 显示编辑弹窗
    document.getElementById('editArticleModal').classList.add('
show');
}

/**
 * 保存文章编辑的函数
 * 该函数会获取编辑后的文章内容，更新到本地存储中，并刷新文章列表
 */
function saveArticleEdit() {
    // 获取当前编辑的文章 ID

```

```

    const articleId =
document.getElementById('editArticleId').value;
    // 从本地存储中获取所有文章数据
    let articles =
JSON.parse(localStorage.getItem('blog_articles'));
    // 查找当前编辑的文章在数组中的索引位置
    const index = articles.findIndex(a => a.id === articleId);

    // 更新文章标题
    articles[index].title =
document.getElementById('editArticleTitle').value;
    // 更新文章内容
    articles[index].content =
document.getElementById('editArticleContent').value;

    // 将更新后的文章数据保存回本地存储
    localStorage.setItem('blog_articles',
JSON.stringify(articles));
    // 显示成功提示
    showToast('文章修改成功', 'success');
    // 关闭编辑模态框
    closeEditModal();
    // 重新加载文章列表
    loadArticleList();
}

```

删除文章:

```

function deleteArticle(articleId) {
    // 显示确认对话框，用户确认后执行回调函数
    showConfirm('确定要删除这篇文章吗?', () => {
        // 从 localStorage 获取并解析文章列表数据
        let articles =
JSON.parse(localStorage.getItem('blog_articles'));
        // 过滤掉指定 ID 的文章，创建新的文章列表
        articles = articles.filter(a => a.id !== articleId);
        // 将更新后的文章列表保存回 localStorage
        localStorage.setItem('blog_articles',
JSON.stringify(articles));

        // 显示删除成功的提示信息
        showToast('文章删除成功', 'success');
        // 重新加载文章列表以更新界面显示
        loadArticleList();
    });
}

```


3.4.3 分类管理

分类管理功能允许用户添加、编辑和删除博客分类。由于每篇文章都必须属于一个分类,分类管理是内容组织的基础。系统提供了完善的分类管理界面和逻辑,包括重名检测、级联更新等高级特性。

添加分类功能:

`addCategory()` 函数负责处理分类的添加。首先,系统从表单获取分类名称和描述,并去除首尾空格。然后从 `localStorage` 读取现有的分类数组。

在添加新分类之前,系统会检查是否已存在同名分类。这个检查使用数组的 `some()` 方法,判断是否有任何分类的 `name` 属性与新分类名相同。如果发现重复,系统会显示警告提示"分类已存在",并中止操作。这个设计防止了重复分类的出现,保持了分类系统的整洁性。

如果检查通过,系统创建一个新的分类对象,包含 `id`、`name` 和 `description` 三个属性。`id` 使用 `Date.now()` 生成,这是一个基于时间戳的唯一值,确保每个分类都有独特的标识符。新分类被添加到数组末尾,然后整个数组被保存回 `localStorage`。最后显示成功提示并刷新分类列表。

修改分类功能:

`editCategory()` 函数处理分类的重命名。这个功能比较复杂,因为修改分类名称后,还需要同步更新所有使用该分类的文章。

首先,系统根据分类 ID 找到目标分类,然后使用 `prompt()` 函数弹出一个输入对话框,显示当前分类名作为默认值,让用户输入新名称。如果用户点击取消或输入为空,则中止操作。

确认修改后,系统需要执行两个更新操作:一是更新分类表中的分类名称,二是遍历所有文章,将原分类名替换为新分类名。这种级联更新确保了数据的一致性,避免了分类名不匹配的问题。

这种设计是必要的,因为文章通过分类名字符串与分类关联,而不是通过分类 ID。这也是为什么修改分类名称需要谨慎的原因。

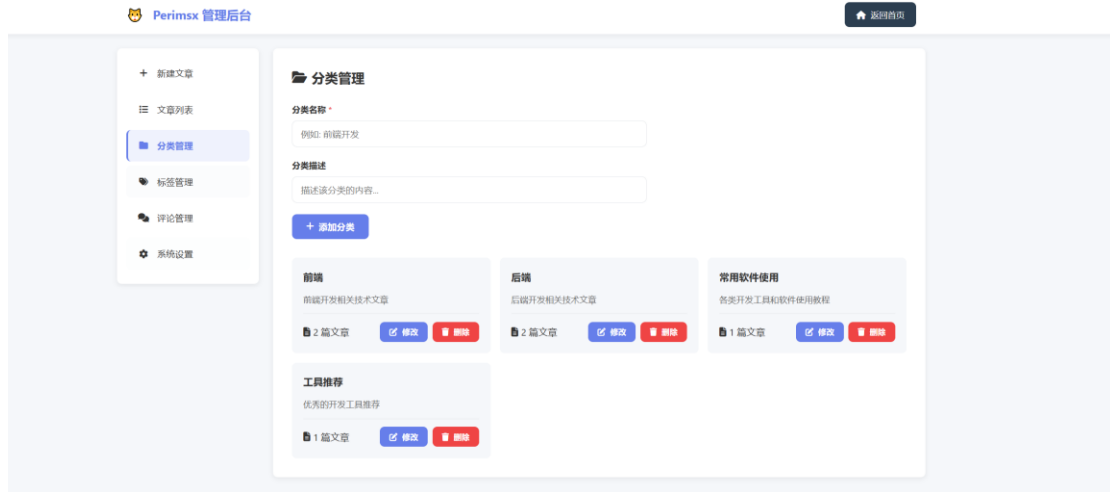
删除分类功能:

`deleteCategory()` 函数处理分类的删除。删除分类之前,系统会检查是否有文章使用了该分类。这个检查通过遍历所有文章实现,如果发现任何文章使用了目标分类,系统会提示"该分类下还有文章,无法删除",并中止操作。

这个设计防止了删除有文章的分类,避免了文章"失去分类"的情况。如果需要删除某个分类,必须先将该分类下的所有文章移动到其他分类或删除。

如果检查通过,系统会弹出确认对话框。用户确认后,使用 `filter()` 方法从分类数组中移除目标分类,然后保存回 `localStorage`。

效果;



添加分类:

```
function addCategory() {
    // 获取并去除分类名称和描述的前后空格
    const name =
document.getElementById('categoryName').value.trim();
    const desc =
document.getElementById('categoryDesc').value.trim();

    // 从 localStorage 获取现有分类列表, 如果没有则初始化为空数组
    let categories =
JSON.parse(localStorage.getItem('blog_categories') || '[]');

    // 检查分类名称是否已存在
    if (categories.some(c => c.name === name)) {
        showToast('分类已存在', 'warning');
        return;
    }

    // 添加新分类到数组
    categories.push({
        id: Date.now(), // 使用时间戳作为唯一 ID
        name: name,
        description: desc
    });

    // 将更新后的分类列表保存到 localStorage
```

```
    localStorage.setItem('blog_categories',
JSON.stringify(categories));
    showToast('分类添加成功', 'success');
    loadCategoryList(); // 重新加载分类列表
}
```

修改分类:

```
function editCategory(categoryId) {
    // 从 localStorage 获取并解析分类数据
    let categories =
JSON.parse(localStorage.getItem('blog_categories'));
    // 根据 ID 查找对应的分类对象
    const category = categories.find(c => c.id === categoryId);

    // 弹出提示框让用户输入新的分类名称
    const newName = prompt('请输入新的分类名称:',
category.name);
    // 如果用户取消输入，则直接返回
    if (!newName) return;

    // 更新分类名称
    category.name = newName;
    // 将更新后的分类数据保存回 localStorage
    localStorage.setItem('blog_categories',
JSON.stringify(categories));

    // 同步更新所有文章的分类
    // 获取并解析文章数据，如果不存在则初始化为空数组
    let articles =
JSON.parse(localStorage.getItem('blog_articles') || '[]');
    // 遍历所有文章，将旧分类名称更新为新名称
    articles.forEach(article => {
        if (article.category === category.name) {
            article.category = newName;
        }
    });
    // 将更新后的文章数据保存回 localStorage
    localStorage.setItem('blog_articles',
JSON.stringify(articles));

    // 显示成功提示消息
    showToast('分类修改成功', 'success');
    // 重新加载分类列表以更新显示
    loadCategoryList();
}
```

```
}
```

3.4.4 标签管理

标签管理功能提供了标签的创建、编辑和删除功能。与分类相比,标签的管理更加灵活,因为一篇文章可以拥有多个标签,而标签可以随时添加和删除。系统的标签管理功能同样支持级联更新,确保数据一致性。

添加标签功能:

`addTag()` 函数负责处理标签的添加。与分类不同,标签数据结构更加简单,存储为一个字符串数组,而不是对象数组。这是因为标签通常只需要名称一个属性,不需要描述等额外信息。

系统从输入框获取标签名称并去除空格,然后从 `localStorage` 读取现有标签数组。在添加新标签之前,同样会检查是否已存在同名标签。这次检查使用数组的 `includes()` 方法,直接判断数组中是否包含该字符串。如果发现重复,显示警告提示并中止操作。

如果检查通过,直接将新标签名添加到数组末尾,然后保存回 `localStorage`。添加成功后显示提示并刷新标签列表。标签的简单结构使得添加操作非常轻量级。

修改标签功能:

`editTag()` 函数处理标签的重命名。与分类修改类似,这个功能也需要级联更新所有使用该标签的文章。

首先使用 `prompt()` 弹出输入对话框,显示当前标签名作为默认值。用户输入新标签名后,系统需要执行两步更新:

1. **更新标签表:** 在标签数组中找到旧标签的索引,使用 `indexOf()` 方法实现,然后将该索引位置的值更换为新标签名。
2. **更新文章标签:** 遍历所有文章,如果文章的 `tags` 数组中包含旧标签,就找到该标签在数组中的位置,然后替换为新标签。这个过程需要处理数组中的数组结构,因为每篇文章的 `tags` 属性本身就是一个数组。

这种级联更新机制确保了当修改标签名称后,所有相关数据都保持同步,避免了标签名不匹配的问题。修改完成后显示成功提示并刷新标签列表。

删除标签功能:

删除标签的逻辑相对简单。由于标签是多对多关系,删除一个标签不会导致文章“失去分类”的问题。系统会弹出确认对话框,用户确认后使用 `filter()` 方法从标签数组中移除目标标签。同时也需要遍历所有文章,从文章的 `tags` 数组中移除该标签。删除完成后保存数据并刷新列表。

效果：



添加标签：

```
function addTag() {
    // 获取标签输入框的值并去除前后空格
    const name =
document.getElementById('tagName').value.trim();

    // 从 localStorage 中获取已存在的标签列表，如果不存在则初始化为空数组
    let tags = JSON.parse(localStorage.getItem('blog_tags') || '[]');

    // 检查标签是否已存在
    if (tags.includes(name)) {
        // 如果标签已存在，显示警告提示并返回
        showToast('标签已存在', 'warning');
        return;
    }

    // 将新标签添加到标签数组
    tags.push(name);
    // 更新 localStorage 中的标签列表
    localStorage.setItem('blog_tags', JSON.stringify(tags));
    // 显示成功提示
    showToast('标签添加成功', 'success');
    // 重新加载标签列表
```

```
loadTagList();  
}
```

修改标签:

```
function editTag(oldTag) {  
  // 弹出提示框让用户输入新的标签名称，默认值为旧标签名称  
  const newTag = prompt('请输入新的标签名称:', oldTag);  
  // 如果用户取消输入，则直接返回  
  if (!newTag) return;  
  
  // 从 localStorage 中获取所有标签，如果没有则初始化为空数组  
  let tags = JSON.parse(localStorage.getItem('blog_tags') ||  
    '[]');  
  // 查找旧标签在标签数组中的索引  
  const index = tags.indexOf(oldTag);  
  // 更新标签数组中的标签名称  
  tags[index] = newTag;  
  // 将更新后的标签数组保存回 localStorage  
  localStorage.setItem('blog_tags', JSON.stringify(tags));  
  
  // 同步更新所有文章的标签  
  // 从 localStorage 中获取所有文章，如果没有则初始化为空数组  
  let articles =  
    JSON.parse(localStorage.getItem('blog_articles') || '[]');  
  // 遍历每篇文章  
  articles.forEach(article => {  
    // 如果文章包含标签数组且旧标签在文章标签中  
    if (article.tags && article.tags.includes(oldTag)) {  
      // 找到旧标签在文章标签数组中的索引  
      const tagIndex = article.tags.indexOf(oldTag);  
      // 更新文章标签数组中的标签名称  
      article.tags[tagIndex] = newTag;  
    }  
  });  
  // 将更新后的文章数组保存回 localStorage  
  localStorage.setItem('blog_articles',  
    JSON.stringify(articles));  
  
  // 显示修改成功的提示  
  showToast('标签修改成功', 'success');  
  // 重新加载标签列表  
  loadTagList();  
}
```


3.4.5 评论管理

评论管理功能允许管理员查看和删除博客中的所有评论。这个功能对于内容审核和垃圾评论管理非常重要。系统采用聚合式设计,将所有文章的评论汇总到一个统一的管理界面中。

评论列表加载逻辑:

`loadAllComments()` 函数负责加载和显示所有评论。由于评论是按文章分散存储的(每篇文章的评论存储在独立的 `localStorage` key 中),需要首先遍历所有文章,收集它们的评论。

系统首先创建一个空数组 `allComments`,然后从 `localStorage` 读取所有文章。对每篇文章,调用 `getComments(article.id)` 获取其评论列表。为了在管理界面中显示评论属于哪篇文章,系统使用展开运算符(`...comment`)复制评论对象,并添加 `articleId` 和 `articleTitle` 两个额外属性,这样每个评论就带有了其所属文章的信息。

收集完所有评论后,系统按照创建时间降序排列,使用 `sort()` 方法比较 `created_at` 属性,让最新的评论显示在最前面。这种排序方式便于管理员优先处理最新的评论。

评论卡片渲染:

评论列表以卡片形式展示,每个评论卡片包含三个部分:

1. **评论头部:** 显示评论者的昵称和所属文章的标题。文章标题以较小的字体显示在昵称后面,帮助管理员快速了解评论的上下文。
2. **评论内容:** 显示评论的正文内容。内容会进行 HTML 转义处理,确保安全显示。
3. **操作按钮:** 提供删除按钮,允许管理员删除不适当的评论。按钮采用红色,明确指示删除操作的破坏性。

卡片的设计简洁明了,采用白色背景和阴影效果,与页面背景区分开来。多个评论卡片垂直排列,之间有适当的间距,提升可读性。

删除评论功能:

`deleteComment()` 函数处理评论的删除。由于删除是不可逆操作,系统首先弹出确认对话框,确保管理员不是误操作。

用户确认后,系统根据文章 ID 获取该文章的所有评论,然后使用 `filter()` 方法过滤掉目标评论(通过评论 `id` 匹配)。过滤后的评论数组被重新保存到 `localStorage`,使用 `comments_${articleId}` 作为 key,覆盖原有数据。

删除成功后,系统显示成功提示并调用 `loadAllComments()` 重新加载评论列表,让删除效果立即显示。这个过程确保了界面与数据的同步。

评论管理特点:

评论管理功能的一个亮点是它的聚合显示能力。管理员不需要逐个打开文章查看评论,而是在一个页面中就能看到所有评论。每个评论都清楚标记了属于哪篇文章,方便管理员进行上下文判断。这种设计大大提升了评论管理的效率。

效果:



评论列表渲染:

```
function loadAllComments() {
  // 用于存储所有评论的数组
  let allComments = [];
  // 从 localStorage 中获取所有博客文章数据，如果没有则使用空数组
  const articles =
JSON.parse(localStorage.getItem('blog_articles') || '[]');

  // 遍历每篇文章
  articles.forEach(article => {
    // 获取当前文章的所有评论
    const comments = getComments(article.id);
    // 遍历评论并将文章信息添加到评论对象中
    comments.forEach(comment => {
      allComments.push({
        ...comment,
        articleId: article.id,
        articleTitle: article.title
      });
    });
  });

  // 按评论创建时间降序排序
  allComments.sort((a, b) =>
    new Date(b.created_at) - new Date(a.created_at));

  // 将评论列表渲染到页面上
  document.getElementById('commentList').innerHTML =
    allComments.map(comment => `
    <div class="comment-card">
      <div class="comment-header">
        <strong>${comment.author}</strong>
        <span>${comment.articleTitle}</span>
      </div>
      <div class="comment-
content">${comment.content}</div>
      <div class="comment-actions">
        <button
onclick="deleteComment('${comment.articleId}',
${comment.id})">删除
      </button>
      </div>
    </div>
    `).join('');
```

```
}
```

删除评论:

```
function deleteComment(articleId, commentId) {  
  // 显示确认对话框, 用户确认后执行删除操作  
  showConfirm('确定要删除这条评论吗?', () => {  
    // 获取当前文章的所有评论  
    let comments = getComments(articleId);  
    // 过滤掉要删除的评论  
    comments = comments.filter(c => c.id !== commentId);  
    // 将更新后的评论列表保存到 localStorage  
    localStorage.setItem(`comments_${articleId}`,  
      JSON.stringify(comments));  
  
    // 显示删除成功的提示  
    showToast('评论删除成功', 'success');  
    // 重新加载所有评论以更新界面  
    loadAllComments();  
  });  
}
```

3. 4. 6 系统设置

系统设置功能允许用户个性化定制博客的外观和配置。这个模块包括网站图标设置、个人头像设置、个人简介设置和密钥修改等功能。所有设置都存储在 localStorage 中, 确保持久化。

网站图标设置:

网站图标 (Logo) 是博客的视觉标识, 显示在导航栏左侧。系统支持两种图标类型: Emoji 和 Font Awesome 图标。

updateSiteIcon() 函数处理图标的设置。首先获取用户选择的图标类型 (emoji 或 fontawesome) 和图标内容。如果选择 Emoji, 用户可以输入任何 Emoji 符号,

比如 🐱、🚀 等。如果选择 Font Awesome, 用户需要输入图标的类名, 比如 fa-

heart。设置信息被保存到 localStorage 的两个 key: site_icon 存储图标内容, site_icon_type 存储图标类型。保存后调用 updateIconPreview() 更新预览, 让用户立即看到效果。toggleIconInput() 函数负责根据选择的图标类型调整输入框的提示信息。当选择 Emoji 时, 标签文字变为 "输入 Emoji", 占位符显示示例; 当选择 Font Awesome 时, 标签文字变为 "输入 Font Awesome 类名", 占位符显示相应示例。这种动态提示帮助用户理解应该输入什么格式的内容。

效果:



个人头像设置:

个人头像显示在侧边栏的个人简介区域。系统支持两种头像设置方式:本地上传和 URL 输入。

`previewAvatar()` 函数处理本地图片的上传和预览。当用户选择图片文件后,系统首先检查文件大小,限制在 2MB 以内。如果超出大小限制,显示警告提示并中止上传。

如果文件大小合适,系统使用 `FileReader` API 读取图片文件。`readAsDataURL()` 方法将图片转换为 Base64 编码的 Data URL 格式,这种格式可以直接嵌入 HTML 中显示,也可以存储在 `localStorage` 中。读取完成后,图片会显示在预览区域,同时 Data URL 被填充到隐藏的输入框中。

`updateAvatar()` 函数保存头像设置。无论是本地上传还是 URL 输入,最终都转换为一个字符串(Data URL 或普通 URL)存储在 `blog_avatar` key 中。这种统一的存储格式使得读取和显示逻辑非常简单。

效果:



个人简介设置:

个人简介显示在侧边栏,用于介绍博主的背景和兴趣。简介字数限制在 200 字以内。

简介输入框绑定了 input 事件监听器,实时统计字符数并显示在输入框下方。当字符数超过 200 时,系统会自动截断超出部分,使用 substring(0, 200)保留前 200 个字符。这种实时限制避免了用户输入过多内容后提交失败的问题。

字符计数显示为 "X/200" 的格式,X 是当前字符数,200 是最大限制。这个视觉反馈帮助用户掌握输入长度,合理控制简介的篇幅。

效果：。



修改密钥：

修改密钥功能允许用户更改后台管理的访问密钥。这是一个重要的安全功能。

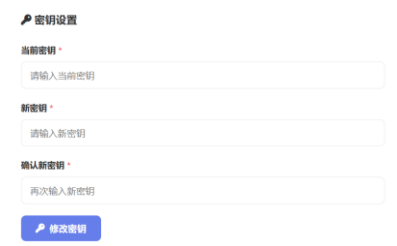
changePassword() 函数处理密钥修改。首先验证当前密钥是否正确,与 localStorage 中存储的 admin_key 比较。如果当前密钥错误,显示错误提示并中止操作。这个验证防止了未授权用户修改密钥。

然后检查新密钥的长度,要求至少 6 位。这是为了确保密钥有一定的复杂度,提高安全性。如果新密钥过短,显示警告提示。

接着检查两次输入的新密钥是否一致。这个检查防止了用户因输入错误而设置错误的密钥。如果两次输入不一致,显示警告提示。

所有验证通过后,将新密钥保存到 admin_key key。保存成功后显示提示,并在 1.5 秒后自动刷新页面。刷新页面会触发身份验证,要求用户使用新密钥重新登录,验证新密钥是否设置成功。

效果：



头像设置:

```
function previewAvatar(event) {
    const file = event.target.files[0]; // 获取用户选择的文件
    if (file && file.size <= 2 * 1024 * 1024) { // 检查文件大小
        // 是否小于等于 2MB
        const reader = new FileReader(); // 创建 FileReader 对象
        // 用于读取文件
        reader.onload = function(e) {
            // 文件读取成功后，将读取的结果设置为头像图片的 src 和
            // avatarUrl 的值
            document.getElementById('currentAvatar').src =
            e.target.result;
            document.getElementById('avatarUrl').value =
            e.target.result;
        };
        reader.readAsDataURL(file); // 将文件读取为 Data URL
    } else {
        // 如果文件大小超过 2MB，显示警告提示
        showToast('图片大小不能超过 2MB', 'warning');
    }
}

/**
 * 更新用户头像
 */
function updateAvatar() {
    const avatarUrl =
    document.getElementById('avatarUrl').value.trim(); // 获取头像
    // URL 并去除首尾空格
    if (!avatarUrl) { // 检查 URL 是否为空
        // 如果 URL 为空，显示警告提示
        showToast('请选择图片或输入 URL', 'warning');
        return; // 终止函数执行
    }
}
```

```

// 将头像 URL 保存到 localStorage
localStorage.setItem('blog_avatar', avatarUrl);
// 显示成功提示
showToast('头像设置成功', 'success');
}

```

个人简介设置:

```

const bioContent = document.getElementById('bioContent');
const bioCharCount = document.getElementById('bioCharCount');

// 为简介内容输入框添加输入事件监听器
bioContent.addEventListener('input', () => {
  // 获取当前输入内容的长度
  const length = bioContent.value.length;
  // 更新字符计数显示
  bioCharCount.textContent = length;

  // 检查输入内容是否超过 200 个字符
  if (length > 200) {
    // 如果超过限制，截取前 200 个字符
    bioContent.value = bioContent.value.substring(0, 200);
    // 更新字符计数为 200
    bioCharCount.textContent = 200;
  }
});

function updateBio() {
  const bio =
document.getElementById('bioContent').value.trim();
  localStorage.setItem('blog_bio', bio);
  showToast('简介保存成功', 'success');
}

```

修改密钥:

```
function changePassword() {  
    // 获取当前密码、新密码和确认密码的输入值  
    const currentPwd =  
document.getElementById('currentPassword').value;  
    const newPwd =  
document.getElementById('newPassword').value;  
    const confirmPwd =  
document.getElementById('confirmPassword').value;  
  
    // 从 localStorage 获取存储的密钥，如果不存在则使用默认值  
'123456'  
    const storedKey = localStorage.getItem('admin_key') ||  
'123456';  
  
    // 验证当前输入的密码是否与存储的密钥匹配  
    if (currentPwd !== storedKey) {  
        showToast('当前密钥错误', 'error');  
        return;  
    }  
  
    // 验证新密码长度是否至少为 6 位  
    if (newPwd.length < 6) {  
        showToast('新密钥至少 6 位', 'warning');  
        return;  
    }  
  
    // 验证两次输入的新密码是否一致  
    if (newPwd !== confirmPwd) {  
        showToast('两次输入的密钥不一致', 'warning');  
        return;  
    }  
}
```



```

// 将新密码保存到 localStorage
localStorage.setItem('admin_key', newPwd);
showToast('密钥修改成功,请重新登录', 'success');

// 设置 1.5 秒后刷新页面, 要求用户重新登录
setTimeout(() => {
    location.reload();
}, 1500);
}

```

3.5 数据存储机制

系统使用 localStorage 实现客户端数据持久化, 主要数据类型包括:

文章数据:

```

// 保存文章列表
localStorage.setItem('blog_articles', JSON.stringify(articles));
// 读取文章列表
const articles = JSON.parse(localStorage.getItem('blog_articles') || '[]');

```

分类标签数据:

```

// 分类
localStorage.setItem('blog_categories', JSON.stringify(categories));

// 标签
localStorage.setItem('blog_tags', JSON.stringify(tags));

```

评论数据:

```

// 按文章 ID 存储评论
localStorage.setItem(`comments_${articleId}`,
JSON.stringify(comments));

```

系统设置:

```

localStorage.setItem('blog_avatar', avatarUrl);
localStorage.setItem('blog_bio', bio);
localStorage.setItem('site_icon', icon);
localStorage.setItem('admin_key', key);

```

浏览量统计:

```

localStorage.setItem(`views_${articleId}`, views);

```

3.6 样式设计

系统采用现代化的设计风格, 主要 CSS 特性包括:

CSS 变量定义:

`* 全局 CSS 变量定义，用于在整个主题中保持颜色一致性 */`

```
:root {  
    /* 主要颜色 - 用于按钮、链接等重要元素 */  
    --primary-color: #3b82f6;  
    /* 次要颜色 - 用于次要元素和装饰 */  
    --secondary-color: #64748b;  
    /* 背景颜色 - 用于页面背景 */  
    --background-color: #f0f7ff;  
    /* 卡片背景色 - 用于卡片式组件的背景 */  
    --card-background: #ffffff;  
    /* 主要文本颜色 - 用于正文内容 */  
    --text-color: #1e293b;  
    /* 边框颜色 - 用于分割线和边框 */  
    --border-color: #e2e8f0;  
}
```

响应式布局:

```
.container {  
    max-width: 1200px;    /* 设置容器的最大宽度为 1200 像素 */  
    margin: 0 auto;       /* 设置容器上下边距为 0，左右边距自动，实  
    现水平居中 */  
    display: grid;        /* 使用网格布局 */  
    grid-template-columns: 1fr 320px; /* 定义两列布局，第一列占  
    据剩余空间，第二列固定宽度为 320 像素 */  
    gap: 2.5rem;          /* 设置网格项之间的间距为 2.5rem */  
}  
  
/* 响应式媒体查询，当屏幕宽度小于等于 900 像素时 */  
@media (max-width: 900px) {  
    .container {  
        grid-template-columns: 1fr; /* 在小屏幕设备上，将布局改为  
        单列，占据全部可用宽度 */  
    }  
}
```

卡片阴影效果:

```

/* 容器样式设置 */
.blog-post {
    background: var(--card-background);
    border-radius: 0.75rem;
    padding: 2rem;
    box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    transition: all 0.2s ease;
}

.blog-post:hover {
    transform: translateY(-5px);
    box-shadow: 0 10px 15px rgba(0,0,0,0.1);
}

```

动画效果:

```

@keyframes slideInUp {
    /* 动画开始状态 */
    from {
        opacity: 0;          /* 初始透明度为 0（完全透明） */
        transform: translateY(30px); /* 初始位置向下偏移 30 像素 */
    }
    /* 动画结束状态 */
    to {
        opacity: 1;          /* 结束时透明度为 1（完全不透明） */
        transform: translateY(0); /* 结束时位置回到原始状态（无偏移） */
    }
}

/* 应用动画到.blog-post 类 */
.blog-post {
    animation: slideInUp 0.6s ease-out; /* 使用 slideInUp 动画，持续 0.6 秒，使用 ease-out 缓动函数 */
}

```

4. 设计总结

本项目成功实现了一个功能完整的轻量级个人博客系统,充分展示了纯前端技术在构建实用 Web 应用方面的强大能力。在技术能力提升方面,对 HTML5 语义化标签的应用有了深入理解,通过合理使用语义化标签,使代码结构清晰,提升了页面的可访问性和 SEO 友好性,还掌握了使用 data 属性存储自定义数据、利用表单验证 API 增强用户体验等技巧;CSS3 布局技术得到全面实践,综合运用 Flexbox 和 Grid 布局实现响应式设计,通过 CSS 变量统一管理主题色彩,利用过渡动画和关键帧动画提升交互体验,深刻理解了 CSS 的层叠规则和盒模型原理;在 JavaScript 编程方面,实现了从数据管理到视图渲染的完整流程,通过面向对象的 BlogSystem 类设计,实现了代码的模块化和可维护性,深入应用 LocalStorage API,理解了客户端数据持久化的原理和最佳实践,运用事件委托、防抖节流等性能优化技巧,提升了系统的运行效率。在系统设计经验方面,数据架构设计上采用 localStorage 作为数据存储方案,通过合理的数据结构设计,实现了文章、分类、标签、评论等实体之间的关联关系,同时进行了容错处理;功能模块设计上遵循单一职责原则,分层设计使系统结构清晰,便于维护和扩展;用户体验设计上注重细节打磨,如搜索功能的实时反馈、评论表单的智能记忆等,通过响应式布局适配不同设备,保证了系统的可用性。但项目仍存在不足之处,localStorage 的容量限制、缺乏数据导入导出功能、Markdown 编辑器简陋、性能优化和安全性问题等,未来可以考虑集成 IndexedDB、对接云存储服务、增加数据导出备份功能、集成专业编辑器、实现虚拟滚动或分页加载、使用加密算法加密存储密钥等。总体而言,本项目是一次成功的 Web 前端综合实践,通过完整的系统设计和开发过程,巩固了基础知识,培养了系统性思维和工程化能力,提升了独立解决问题的能力,也认识到优秀的 Web 应用需要扎实的技术功底、良好的产品思维 and 用户体验意识,未来 Web 技术仍在快速发展,本项目的设计思想和开发方法对学习现代前端框架具有重要基础作用,通过这次实验,更加坚定了深入学习 Web 开发技术的决心,期待在未来的学习和实践中不断精进,创造出更多有价值的应用。

参考文献

[1] 王艳阁,邹迎,周明. Web 开发技术及实战. 哈尔滨工程大学出版社,2024.06.